A Lock-Free SPSC Ring Buffer for Real-Time Wikipedia Edit Stream Processing

Raj Mhetar

May 2026 – September 2026

Abstract

This paper describes the design, implementation, and measurement of a real-time stream-processing pipeline that consumes the live Wikipedia edit feed and detects anomalous bursts of editing activity. The core of the system is a header-only, lock-free single-producer/single-consumer (SPSC) ring buffer written in C++20, built to hand events from a network thread to analysis threads without ever taking a lock. The queue is optimized in three independent ways: acquire/release memory ordering instead of sequential consistency, cache-line padding to eliminate false sharing between the producer and consumer cursors, and cached copies of the opposing cursor to avoid touching the other thread's cache line on the common path. Against a mutex-protected `std::deque` baseline the optimized queue sustains 20× the throughput (164.0 vs. 8.1 million operations per second) and cuts 99.9th-percentile handoff latency by a factor of 13.75 (400 ns vs. 5500 ns). Measuring the complete pipeline, however, showed that the stage that reads and parses each JSON payload costs roughly 900× more per event than a queue handoff, so the end-to-end system runs at 165 K events/s and the queue optimization is invisible at the system level. Against a live stream measured at 43.2 events/s, the pipeline has roughly three orders of magnitude more capacity than the workload requires. The most valuable outcomes of the project were therefore methodological: an unoptimized build hid the entire effect of the optimization, profiling the whole pipeline revealed that the component I had tuned was never the bottleneck, and a later review of the benchmark harness found that it had not been measuring quite what I thought it was.

1 Introduction

1.1 The problem

Wikipedia publishes a live, never-ending feed of every edit made to every one of its language editions. I wanted to build a program that watches this feed and raises an alert when a page suddenly starts being edited far faster than usual — the signature of breaking news, an edit war, or vandalism in progress.

Doing this in real time has a structural difficulty. One part of the program must sit on a network connection and pull bytes off the socket as fast as they arrive; if it falls behind, the server disconnects. Another part must do the actual analysis, which is slower and whose cost varies from event to event. These two jobs have to run on separate threads, and something has to carry events from one to the other. That "something" is a queue, and every event in the system passes through it. The question this project set out to answer is: *how should that queue be built, and how much does the answer matter?*

1.2 What I did

I built the complete pipeline in C++20: a network client that reads the feed, a parser that turns each JSON payload into a fixed-size record, a router that spreads records across four analysis threads, and a detector that flags anomalous edit bursts. At the center of it I wrote a *lock-free* queue — one that lets the two threads communicate without either ever having to stop and wait for the other — and applied three optimizations to it that are standard in high-performance systems but rarely explained from first principles: relaxed memory ordering, cache-line padding, and cursor caching. Section 2 builds up the vocabulary needed to understand those; Section 3 describes the system.

I then measured it three ways (Section 4): the queue in isolation against two simpler alternatives, the full pipeline end to end, and the real live feed. The first measurement says the optimized queue is 20× faster than the obvious mutex-based design. The second says that difference is invisible in the finished system, because parsing the JSON is about 900 times more expensive than handing off the result. The third says the finished system has around a thousand times more capacity than the live feed needs.

1.3 What went wrong, and what I learned

The results are the smaller half of the story. Section 5 describes the things that went wrong along the way: a benchmark that showed the optimization did nothing because the build was not optimized, a detector that fired constantly until three separate guards were added, and a benchmark harness that — as I only found while reviewing this paper — pinned its two threads to the same physical core and so never exercised the hardware behavior the optimization targets. Section 6 collects the lessons and lays out what I would do differently.

1.4 Contributions

- A header-only, statically-checked SPSC ring buffer with the three optimizations above, and an explanation of each aimed at readers with no background in concurrency.
- A four-shard pipeline around it, with a title-hashed router that needs no cross-thread coordination and a drop-on-full backpressure policy chosen for the network context.
- A capture/replay abstraction that makes an uncontrollable live feed deterministic, enabling both tests and reproducible benchmarks.
- Measurements at three levels of the system, and an honest account of where the measurements were wrong or measured less than they appeared to.

2 Background

This section assumes no prior familiarity with the project or with concurrent programming. It builds up the problem from first principles.

2.1 Streaming data and the producer/consumer problem

Wikipedia publishes every edit made across all of its language editions as a public, continuous feed. A program that wants to analyze this feed in real time faces a structural problem: the work of *receiving* data and the work of *analyzing* data have very different characteristics.

Receiving is I/O-bound and latency-sensitive. A single thread holds an open HTTP connection, and if that thread stops reading from the socket, the operating system's receive buffer fills, the TCP window closes, and the server either throttles or disconnects. Analysis, by contrast, is CPU-bound and bursty: some events are cheap to process and some trigger expensive bookkeeping.

The standard solution is to split these into separate threads connected by a queue. The receiving thread (the *producer*) does the minimum work needed to turn bytes into a structured record and immediately hands it off. One or more analysis threads (*consumers*) pull records from the queue and do the expensive work. The queue absorbs the mismatch between the two rates.

This makes the queue itself the critical component. Every single event passes through it, so its cost is paid on every event, and any time the producer spends waiting on the queue is time it is not reading from the socket.

2.2 Why not just use a mutex?

The obvious implementation is a `std::deque` guarded by a `std::mutex`: lock, push, unlock. This is correct and easy to reason about, but it carries costs that matter at high event rates.

A mutex is a mechanism for *mutual exclusion*: it guarantees only one thread touches the data at a time. When the producer holds the lock and the consumer wants it, the consumer must wait. If the wait is long enough, the operating system deschedules the waiting thread and reschedules it later, and this round trip through the kernel scheduler costs microseconds — an eternity relative to the nanoseconds the actual queue operation takes.

Worse, the cost is unpredictable. Most lock acquisitions are uncontended and fast; occasionally one collides with the other thread and takes a thousand times longer. This produces bad *tail latency*: the average looks fine while the worst case is terrible. For a system meant to react in real time, the tail is often what matters.

2.3 The SPSC special case

The general problem — many threads pushing and many popping — is hard to solve without locks. But a much easier special case exists: exactly one thread pushes and exactly one thread pops. This is **single-producer/single-consumer**, or SPSC.

The restriction buys a great deal. With one writer and one reader, the queue can be a fixed-size array with two cursors: a *head* index that only the producer advances, and a *tail* index that only the consumer advances. Because each cursor has exactly one writer, no *read-modify-write* operation is needed — no compare-and-swap, no atomic increment, no lock. Each side reads its own cursor with a simple load and advances it with a simple store. (These are still *atomic* loads and stores, so the other thread can never see a half-written value, but they are the cheapest kind of atomic operation there is.) The only cross-thread communication is each side occasionally reading the other's cursor to learn whether the queue is full or empty.

The array is used circularly: when a cursor runs off the end it wraps back to the beginning, which is why the structure is called a **ring buffer**. Since the cursors only ever increase, "how full is the queue" is simply $head - tail$, and the physical slot for a given cursor value is found by wrapping that value into the array's bounds.

2.4 The two hardware realities that shape the design

Writing a correct lock-free queue requires confronting two facts about how modern processors actually work.

Reordering and memory ordering. Neither the compiler nor the CPU executes memory operations in the order written. Both aggressively reorder for speed, and this is invisible in single-threaded code because both are careful to preserve the appearance of sequential execution *from the perspective of the thread doing the work*. Another thread watching from outside gets no such guarantee.

This breaks the queue. The producer writes an event into a slot and then advances *head* to publish it. If those two writes become visible to the consumer in the opposite order, the consumer sees an advanced *head*, concludes an event is ready, and reads a slot that has not been written yet.

The fix is to constrain the ordering explicitly. C++ exposes this through *memory orderings* on atomic variables. The relevant pair here is **release** and **acquire**. A release store acts as a one-way barrier: everything the thread wrote before it is guaranteed visible to any thread that performs an acquire load and observes that store. Pairing a release store on *head* with an acquire load of *head* guarantees that seeing the new cursor value implies seeing the slot contents.

The alternative — and the C++ default — is **sequential consistency**, which additionally forces every thread to agree on a single global order of all atomic operations. That is a much stronger and more expensive guarantee, and on x86 it requires the compiler to emit a full memory fence on stores. The queue does not need it: it only needs the one-directional guarantee that publishing the cursor publishes the data.

Cache lines and false sharing. CPUs do not move individual bytes between memory and their caches. They move fixed-size blocks called **cache lines**, 64 bytes on all mainstream x86 and ARM processors. Cores keep private copies of lines, and hardware keeps those copies coherent: when one core writes to a line, every other core's copy of that line is invalidated and must be re-fetched.

Coherence operates on whole lines, not on variables. If the producer's *head* and the consumer's *tail* happen to sit in the same 64-byte line — which they will, if declared adjacently — then every time the producer advances *head*, it invalidates the line the consumer is using for *tail*, and vice versa. The two threads never touch the same variable, yet they fight over the same line on every operation. This is **false sharing**, and it converts an intended lock-free fast path into a stream of cache-coherence traffic.

One detail of this picture matters later. Modern Intel and AMD processors run two hardware threads on each physical core (*simultaneous multithreading*, or SMT — Intel calls it Hyper-Threading). Two threads sharing a physical core also share its private caches, so between those two threads there is no cross-core coherence traffic at all. False sharing is a phenomenon between *physical cores*; measuring it requires the producer and consumer to actually be on different ones.

2.5 The data source

The pipeline consumes the Wikimedia EventStreams `recentchange` feed, delivered over **Server-Sent Events** (SSE). SSE is a long-lived HTTP response that never terminates; the server writes newline-delimited records as they occur. Each record carries a `data:` line holding a JSON object describing one change: the page title, the wiki, the user, a comment, a timestamp, revision identifiers, the byte-length change, a bot flag, and a block of metadata. A typical record is around 1.3 KB of JSON.

Two properties of this source shaped the project. It is *unbounded* — there is no end to read toward, so the program must be structured around continuous flow and explicit shutdown. And it is *uncontrollable* — the rate and content are whatever Wikipedia's editors happen to be doing, which makes reproducible testing and benchmarking impossible without a way to capture and replay it.

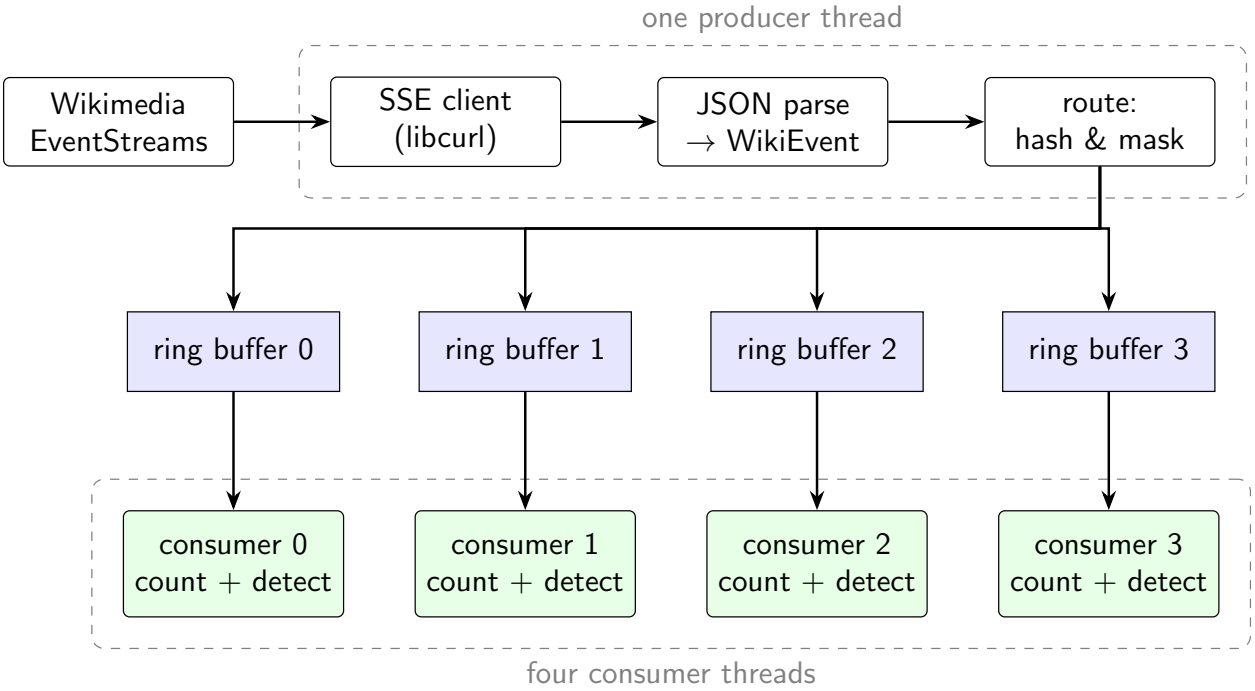

Figure 1: Pipeline architecture. A single producer thread receives, parses, and routes; four independent SPSC ring buffers each feed exactly one consumer thread. Because each queue has one writer and one reader, no lock is required anywhere on the data path.

3 System Design and Implementation

3.1 System overview

The system is a four-stage pipeline with a fan-out in the middle (Figure 1). One producer thread owns the network connection and the parser. It converts each SSE payload into a fixed-size record, chooses a destination shard by hashing the page title, and pushes the record into that shard's queue. Four consumer threads each own one shard exclusively, so every queue has exactly one producer and exactly one consumer, satisfying the SPSC constraint.

3.2 Event representation

Each event is stored in a `WikiEvent` struct built entirely from plain-old-data members: fixed-size character arrays for the title, wiki, user, comment, and type; integers for the timestamp, revision identifiers, and byte-length delta; and a boolean bot flag. Strings longer than their array bound are truncated and null-terminated by the parser.

The fixed-size choice is deliberate and is what makes the queue simple. Because the struct is *trivially copyable*, it can be copied into a queue slot with a plain assignment and requires no destructor, no allocation, and no pointer chasing. Storing `std::string` members instead would put heap pointers in the queue, which would mean the producer allocating and the consumer freeing on every event — reintroducing, through the allocator, exactly the kind of shared mutable state the lock-free queue exists to avoid. The ring buffer enforces this statically with a compile-time assertion that its element type is trivially copyable.

The cost is memory. `WikiEvent` is 848 bytes, so a 1024-slot queue is approximately 848 KB and

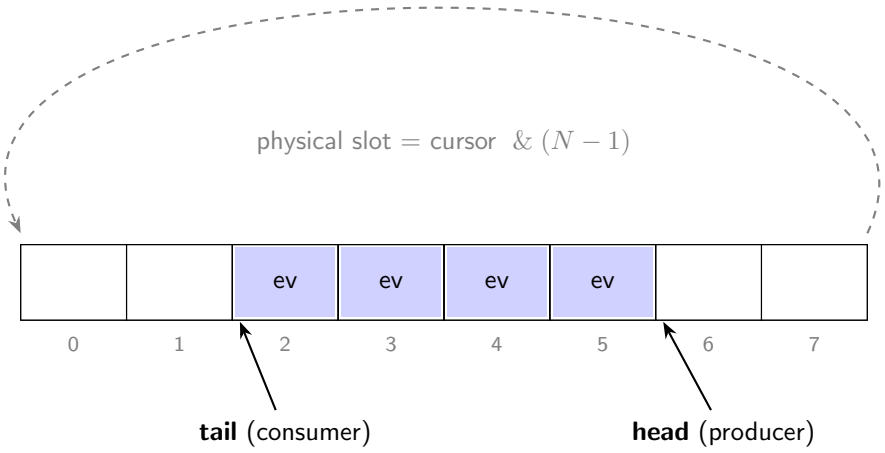

Figure 2: Ring buffer indexing with $N = 8$. The producer writes at *head* and the consumer reads at *tail*; both cursors increase without bound and are masked into the array. Occupancy is $head - tail$, which stays correct across wraparound.

the application's four-shard configuration reserves about 3.3 MB up front. (The pipeline benchmark in Section 4 uses larger 4096-slot queues, about 13.6 MB in total.) This is a deliberate trade: a fixed, predictable memory footprint allocated once, in exchange for never allocating on the data path. One practical consequence is that the shard array must live in static storage rather than on the stack; at these sizes it would overflow the default thread stack on Windows.

3.3 Parsing

The parser is the stage the results turn out to hinge on, so it is worth stating exactly what it does. Each SSE payload is handed to the nlohmann/json library, which parses the *entire* JSON document into an in-memory tree (a "DOM" parse). From that tree the parser extracts nine fields into the `WikiEvent`. For each of the five string fields it first materializes a temporary `std::string` — a heap allocation — and then copies its bytes into the fixed array and discards it.

This is the simplest correct implementation and was chosen for that reason. It is also doing far more work than the pipeline needs: it builds a tree for a document of which only nine fields are ever read, and it allocates five times per event on the producer thread, the one thread whose time is most precious. Section 4.4 measures what this costs.

3.4 Ring buffer design

The queue is a header-only C++ class template parameterized on element type and capacity, with three optimizations layered on the basic structure.

Power-of-two capacity and mask indexing. Capacity is constrained at compile time to a power of two. Mapping a monotonically increasing cursor to a physical slot then becomes a bitwise AND with $N - 1$ rather than a modulo operation, replacing an integer division with a single-cycle instruction (Figure 2). Because the cursors only increase and are never wrapped themselves, the difference $head - tail$ remains a valid occupancy count even across wraparound.

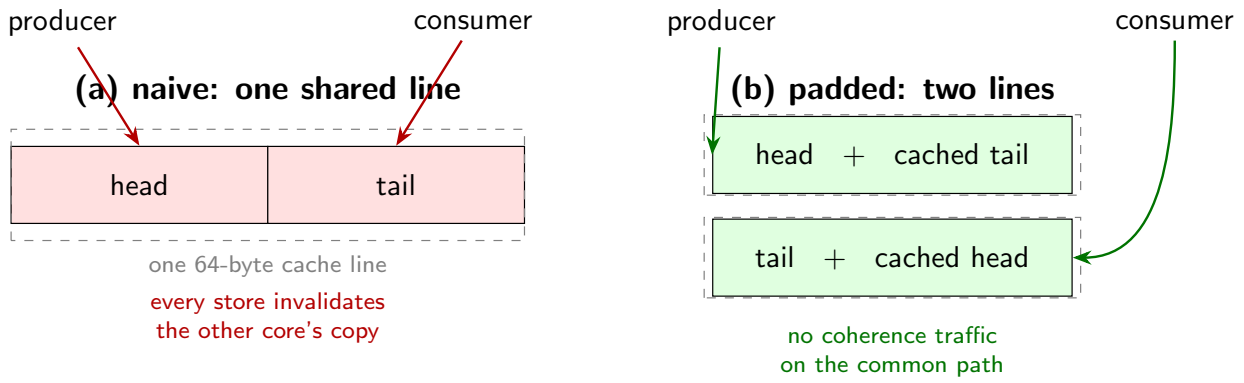

Figure 3: False sharing and its elimination. In (a) the two cursors share a cache line, so the cores contend on every operation despite never touching the same variable. In (b) each side owns a full line containing its cursor and a cached copy of the other's, so steady-state operation generates no cross-core invalidations.

Acquire/release ordering. Both cursors are atomic, but neither uses the default sequentially-consistent ordering. Each side loads *its own* cursor with relaxed ordering, since it is the only writer and needs no cross-thread guarantee to read what it itself wrote. The publishing store is a release, and the load of the *opposing* cursor is an acquire. This is exactly the guarantee the queue needs — that observing a cursor advance implies observing the slot write that preceded it — and nothing more.

Cache-line padding and cursor caching. The producer's and consumer's hot data are placed in separate structures, each aligned to a 64-byte boundary so they occupy distinct cache lines (Figure 3). This directly addresses the false-sharing problem described in Section 2.4.

Padding alone is not sufficient, because each side still has to read the other's cursor to test for full or empty — and that read pulls in the other core's line. The second half of the optimization is that each side keeps a *cached copy* of the opposing cursor alongside its own. The producer checks fullness against its cached tail value; only if that stale value says "full" does it pay to read the real one. Since a stale tail can only underestimate available space, this is always safe: the cached value can cause an unnecessary refresh but never a false "not full" verdict. In steady state where the queue is neither full nor empty, neither thread touches the other's cache line at all.

Notably, the implementation uses a hard-coded 64-byte constant rather than `std::hardware_destructive_int` The standard constant is theoretically more correct but GCC warns against using it in a fixed struct layout, because its value can change with tuning flags and thereby silently change the ABI of any type built around it.

3.5 Sharding

A single SPSC queue admits only one consumer, which caps analysis throughput at one core. To scale, the producer hashes each event's page title and routes it to one of four independent queues, each with its own consumer thread. The shard count is a power of two so routing is a bitmask rather than a modulo, and this is enforced with a compile-time assertion.

Hashing on *title* rather than round-robin is the important design choice. The downstream analysis is per-page, so all events for a given page must reach the same consumer for its state to be complete. Title-hashing guarantees this while requiring no coordination between consumers at all: each owns a disjoint subset of pages, so their detector state is entirely thread-local and needs no

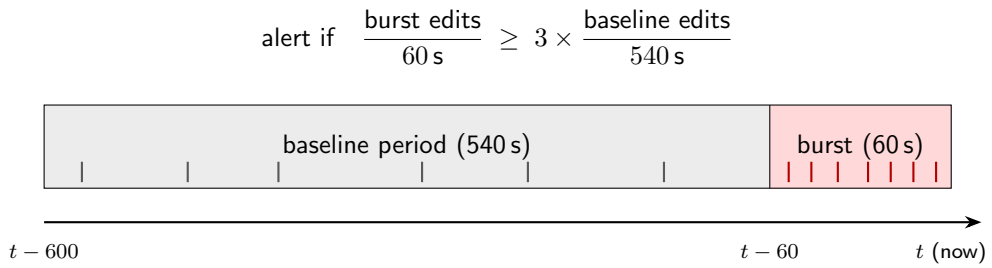

Figure 4: Edit-storm detection windows. Each page's timestamps are kept in a sorted deque spanning ten minutes and split at the one-minute mark by binary search. The recent rate is compared against the page's own prior rate, so the threshold adapts per page rather than being a fixed global count.

synchronization. The cost is that load balance depends on the hash distribution and on the actual popularity distribution of pages, which is heavily skewed.

3.6 Backpressure policy

If a consumer falls behind and its queue fills, the producer does not block — it discards the event and increments a per-shard drop counter. This is a deliberate choice specific to the network context. Blocking the producer would stall the thread reading the socket, which backs up the TCP receive buffer and eventually causes the server to throttle or drop the connection. Dropping an event costs one data point; stalling the reader risks the entire stream. The drop counters are reported alongside the event totals so the loss rate is always visible rather than silent.

3.7 Edit-storm detection

Each consumer runs an independent `EditStormDetector` that flags pages being edited far faster than their own recent norm — a pattern typical of breaking-news activity, edit wars, or vandalism.

For each page the detector keeps a deque of edit timestamps spanning a ten-minute window, split at the one-minute mark (Figure 4). Edits in the last minute form the *burst* rate; the preceding nine minutes form the *baseline* rate. Because events arrive in time order the deque is always sorted, so the split point is found with a binary search rather than a scan. An alert fires when the burst rate reaches three times the baseline rate.

Three guards keep the false-positive rate manageable. Each exists because the detector without it misbehaved in a specific way during development (Section 5):

- **Minimum burst count (5 edits).** Without it, a page edited twice in a minute against a near-zero baseline produces an enormous ratio and a meaningless alert.
- **Minimum baseline count (3 edits).** A brand-new page has no history, so its baseline rate is zero and *any* activity is infinitely larger. This guard means a page cannot alert on its own creation flurry — there must be a prior rate for the burst to be a multiple of.
- **Alert cooldown (30 seconds).** A genuine sustained storm satisfies the condition on every subsequent edit. The cooldown reports it once rather than hundreds of times.

Memory is bounded by evicting pages with no edits for five minutes, run once per minute. Without this, tracking state accumulates for every page ever seen and grows without limit over a long run.

3.8 Capture and replay

A live network stream is a poor foundation for testing and benchmarking: it cannot be replayed, its rate cannot be controlled, and its content is never the same twice. To make the system reproducible, both the live client and a file reader implement a common `EventSource` interface, so the rest of the pipeline cannot tell which one it is driving.

The live client optionally tees every raw JSON payload to a capture file, tab-separated with a microsecond receive timestamp. The replay source reads these files back and reproduces the original inter-event gaps, divided by a speed factor — or, in a special no-delay mode, emits everything as fast as possible. This single abstraction serves three purposes: deterministic integration tests that need no network, benchmarks that run against real traffic patterns, and accelerated replay for exercising the detector against hours of activity in seconds.

3.9 Testing and build infrastructure

The project builds with CMake and is tested with Catch2. All targets compile under strict warnings (`/W4` on MSVC; `-Wall -Wextra -Wpedantic -Wconversion -Wshadow` otherwise). The ring buffer, event struct, parser, detector, and replay source are all header-only, so tests link against them without any library dependency.

4 Results

4.1 What was measured, and how

Three separate measurements were taken, each isolating a different layer:

1. **Queue throughput.** A producer thread pushes as fast as it can while a consumer pops, for a fixed three-second window; the reported figure is the number of items the consumer successfully retrieved, divided by elapsed time. The producer drops on full rather than blocking, so the measurement reflects sustainable end-to-end rate rather than the producer's solo push rate. Both threads are pinned to fixed logical processors so results are not perturbed by the scheduler migrating them. (Which logical processors, and why it matters, is discussed in Section 5.)

2. **Queue latency.** One-way handoff time is measured by timestamping each item as it enters the queue and again as it is popped. The queue capacity is deliberately set to 2, keeping it near-empty so the measurement captures handoff cost rather than time spent waiting in a backlog. The producer re-stamps on every failed push attempt, so the recorded timestamp reflects when the item actually entered the queue, not when the first attempt began. The first 50,000 samples are discarded as warmup and the remaining one million are sorted to extract percentiles.

3. **End-to-end pipeline throughput.** A synthetic capture of 200,000 events across 1,000 distinct page titles is first replayed through the ingest-and-parse stage alone (file read, line splitting, and JSON parse, with no queue) to establish a baseline, then through the complete four-shard pipeline with 4096-slot queues. The difference isolates what the queueing layer adds on top of parsing. The synthetic payloads are deliberately minimal: 182 bytes and nine fields each, versus roughly 1,300 bytes and several dozen fields for a real event.

Three queue implementations are compared: the optimized ring buffer; a *naive* SPSC ring buffer using sequentially-consistent atomics with its cursors adjacent in one cache line; and a

bounded `std::deque` behind a single `std::mutex`. Both baselines are simpler designs that the optimized queue is expected to beat; neither is an established third-party SPSC queue such as `boost::lockfree::spsc_queue`. That comparison is future work.

Correctness is measured separately by a suite of 30 Catch2 test cases, all of which pass. These are summarized in Table 6. The hardware and toolchain used for every measurement are given in Table 1.

Table 1: Measurement environment.

Component	Detail
CPU	Intel Core i9-13900H: 6 performance cores (12 threads) + 8 efficiency cores, 2.6 GHz base
Operating system	Windows 11 (build 26200)
Compiler	GCC 13.2.0 (MSYS2 UCRT64), C++20
Build type	CMake `Release` (−O3, `NDEBUG`)
Dependencies	libcurl, nlohmann/json 3.11.3, Catch2 v3.6.0

4.2 Queue throughput

Table 2 and Figure 5 give the throughput results. The optimized ring buffer sustains 164.0 million operations per second: 4.8× the naive lock-free version and 20.2× the mutex-protected queue.

Table 2: Sustained queue throughput. Capacity 4096, 32-bit integer elements, three-second run, threads pinned.

Implementation	Throughput (Mops/s)	Relative
Mutex + `std::deque`	8.1	1.0×
Naive SPSC (seq_cst, shared cache line)	34.2	4.2×
Optimized SPSC (acq/rel, padded, cached)	**164.0**	**20.2**×

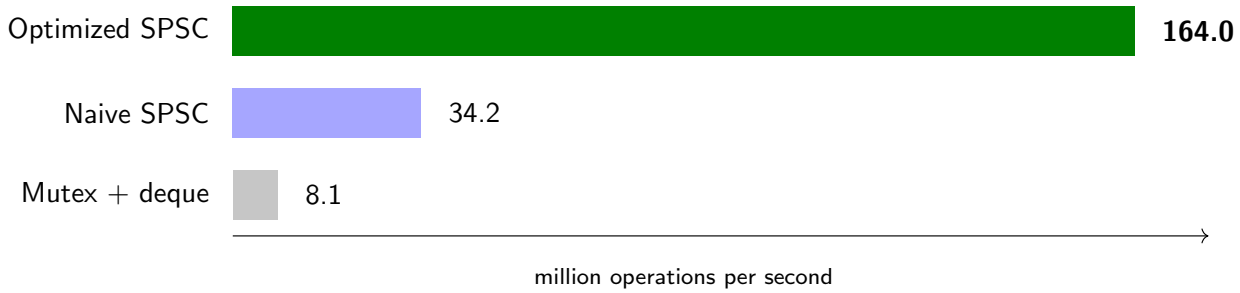

Figure 5: Sustained throughput by queue implementation (higher is better).

4.3 Queue latency

Throughput describes capacity; latency describes responsiveness. The latency results in Table 3 show the more consequential difference. At the median, the lock-free queue is only about twice as fast as the mutex. At the 99.9th percentile it is 13.75× faster.

This is the shape predicted in Section 2.2. The mutex is usually uncontended and therefore fast, but its rare contended cases involve the kernel scheduler and cost microseconds. The lock-free

Table 3: One-way handoff latency, in nanoseconds. Capacity 2, one million measured samples after 50,000 warmup samples, threads pinned.

Implementation	p50	p99	p99.9
Optimized SPSC	**100**	**200**	**400**
Mutex + `std::deque`	200	1400	5500
Improvement	2.0×	7.0×	13.75×

queue has no such cliff: its p99.9 is only four times its median, whereas the mutex's p99.9 is 27× its own median. For a system whose value lies in reacting promptly, this bounded tail is the more meaningful result of the two tables.

One caveat limits how far these numbers can be read. The Windows steady clock advances in 100 ns ticks, and every measured percentile is an exact multiple of 100 ns. The optimized queue's median of 100 ns is therefore at the resolution floor — it means "one tick or less," and the true value may be considerably lower. The throughput measurement supports this: 164.0 Mops/s implies roughly 6 ns per operation, so the real median handoff is likely an order of magnitude below what the clock can resolve. The mutex's tail figures, being far above the tick size, are unaffected by this limitation.

4.4 End-to-end pipeline

Table 4 measures the full path. Reading and parsing 200,000 events with no queueing at all runs at 177 K events/s. Adding the complete four-shard pipeline yields 165 K events/s — an overhead of about 8%, with zero events dropped.

Table 4: End-to-end pipeline throughput, 200,000 synthetic 182-byte events across 1,000 distinct titles, 4096-slot queues.

Configuration	Elapsed (s)	Throughput	Dropped
Ingest + parse only (no queue)	1.128	177 K ev/s	—
Full 4-shard SPSC pipeline	1.216	165 K ev/s	0

This is the most informative result in the project, because it undercuts the premise of the optimization. At 177 K events/s, the ingest-and-parse stage costs roughly 5,650 ns per event. That figure is not the JSON parser alone — it also includes reading the line from the file, splitting off the timestamp, and the five per-event string allocations described in Section 3.3 — but the DOM parse and its allocations are the only parts of it that do more than trivial work. A queue handoff costs roughly 6 ns. **The stage that produces an event is approximately 900 times more expensive than the queue operation that carries it.** Making the queue 20× faster therefore cannot move the system-level number meaningfully — and it does not.

Two things should be said about that 900. First, it was measured on 182-byte synthetic payloads. Real payloads are about seven times larger and have several times as many fields, and a DOM parser's cost scales with both, so on real traffic the ratio is larger still. Second, the claim that the queue is invisible at the system level is inferred from two separate benchmarks rather than demonstrated directly; the direct experiment — running the same pipeline with the mutex queue substituted — is a template-parameter change that I did not make, and is listed as future work.

The 8% overhead the pipeline does show is larger than a 6 ns handoff would suggest, and the reason is instructive. The queue benchmark moves 4-byte integers; the pipeline moves 848-byte

`WikiEvent` structs, copied once on push and again on pop. At roughly 1.7 KB of copying per event plus five threads competing for cores, the measured cost of about 440 ns per event is consistent with memory bandwidth, not synchronization. The lock-free design is not what the pipeline overhead consists of.

4.5 Live stream measurement

To establish what the system actually needs to handle, the live Wikimedia feed was recorded for 30 seconds using the capture facility. It delivered 1,296 events, an average of **43.2 events per second**, in a 1.6 MB capture averaging roughly 1,293 bytes of JSON per event. Replaying that captured real-world traffic through the Release pipeline at maximum speed processed all 1,296 events with zero drops. This is a single 30-second window; the feed's rate varies with time of day and with what is happening in the world, and a longer capture would give a better estimate of both the average and the peaks.

Set against the 165 K events/s the pipeline sustains on synthetic events, the live workload uses roughly 1/3800 of available capacity. Because real payloads are about seven times larger, the true headroom is lower: if parse cost scaled linearly with payload size, the pipeline would sustain on the order of 25 K real events/s, still about 600× the observed rate. Either way, the system has *roughly three orders of magnitude* of headroom over the load it was built for.

4.6 The effect of build configuration

The benchmarks were initially run against a build tree where `CMAKE_BUILD_TYPE` had never been set. CMake leaves this empty by default, which means no optimization flags at all. Table 5 compares those numbers against a proper Release build.

Table 5: Queue throughput (Mops/s) under different build configurations.

Build configuration	Naive	Optimized	Speedup
Empty `CMAKE_BUILD_TYPE` (no −O flags)	25.7	27.2	1.06×
`Release` (−O3)	34.2	164.0	**4.80**×

In the unoptimized build the optimized queue is 6% faster than the naive one — a difference indistinguishable from noise, and one that would reasonably support the conclusion that the whole optimization was pointless. The same code compiled with `-O3` is 4.8× faster. The parse-only benchmark shows the same distortion, rising from 28 K to 177 K events/s, a 6.3× difference.

The explanation is that without optimization the compiler emits redundant loads and stores around every operation, and that overhead is large enough to swamp the memory-ordering and cache-locality effects being measured. The unoptimized build measures the compiler's un-elided bookkeeping, not the data structure.

4.7 Correctness

Two of these deserve specific mention. The ring buffer's threaded test runs a real producer and consumer through 100,000 items and asserts that every item arrives exactly once and in order — the only test that can catch a memory ordering error, since single-threaded tests pass regardless of the orderings chosen. The detector tests drive synthetic timestamps rather than wall-clock time, so time-window behavior spanning ten minutes is tested deterministically in milliseconds, with each guard verified by a case constructed to trip it.

Table 6: Test suite composition. All 30 cases pass.

Suite	Coverage	Cases
Ring buffer	FIFO order, full/empty edges, wraparound, threaded handoff	10
Event parser	Field extraction, truncation, missing fields, malformed JSON	6
Storm detector	Threshold, both guards, cooldown, window and LRU eviction	8
Capture / replay	Ordering, comments, malformed lines, field fidelity, `stop()`	6
Total		**30**

4.8 Threats to validity

- **The two benchmark threads share a physical core.** The harness pins the producer to logical processor 0 and the consumer to logical processor 1. On the i9-13900H those are the two SMT threads of the *same* performance core, so they share L1 and L2 cache. The cross-core coherence traffic that false sharing consists of (Section 2.4) therefore never occurs in the measurement, and the 4.8× naive-to-optimized gap was produced by some combination of memory-ordering cost and intra-core effects rather than by the mechanism Figure 3 depicts. Section 5 discusses how this was found; the fix is a one-line change and a re-run.
- **The naive baseline conflates two variables.** It differs from the optimized queue in both memory ordering *and* cache-line layout simultaneously. The 4.8× gap therefore cannot be attributed between the two; separating them would require building each variant independently.
- **No state-of-the-art baseline.** Both comparison queues were written for this project to be simpler than the optimized one. The results show the optimized queue is much faster than a naive design; they do not show how it compares to a mature SPSC queue such as Boost's or Folly's.
- **The system-level conclusion is inferred, not measured.** The pipeline benchmark was run only with the optimized queue. The claim that a mutex would perform equally well end to end follows from arithmetic on two separate benchmarks, not from a direct comparison.
- **Single runs.** Each configuration was measured once, with no repetition, confidence intervals, or control over background system activity.
- **Clock resolution.** As noted, the sub-microsecond latency figures are quantized to the 100 ns platform tick and understate the lock-free queue's true advantage at the median.
- **Synthetic pipeline workload.** The 200,000-event benchmark uses 182-byte payloads — about one-seventh the size of a real event — and 1,000 uniformly distributed titles. Real payloads make the parser slower, and real Wikipedia traffic is heavily skewed toward popular pages, which would distribute less evenly across shards. Both effects make the synthetic throughput an overestimate.
- **Short live capture.** The 43.2 events/s figure is one 30-second sample of a feed whose rate varies through the day.
- **One platform.** All results are from a single machine, compiler, and operating system. Memory-ordering costs in particular are architecture-dependent; on a weakly-ordered platform such as ARM, the acquire/release versus sequentially-consistent gap would likely be wider than measured here on x86.

5 What Went Wrong Along the Way

Most of what this project taught me came from things that did not work the first time. This section records them in roughly the order they happened.

The first benchmark said the optimization did nothing. The first time I ran the queue benchmark, the optimized ring buffer came out 6% faster than the naive one (Table 5, first row). That is well within run-to-run noise, and the reasonable conclusion was that acquire/release ordering, cache-line padding, and cursor caching were not worth their complexity. The actual cause was that `CMAKE_BUILD_TYPE` had never been set, so the compiler was emitting fully unoptimized code in which every variable access went through memory and the bookkeeping swamped the effect under test. Rebuilding with `-O3` turned the 6% into 4.8×. The reproduction instructions in Appendix A now set the build type explicitly, and the build type is listed in Table 1 as part of the measurement environment rather than as an afterthought.

The detector fired on everything. The first version of the edit-storm detector had just the ratio test: alert if the last minute's rate is three times the previous nine minutes'. Run against live traffic, it alerted almost continuously, for three distinct reasons that each needed its own fix. Newly created pages had a baseline of zero, so their first few edits produced an infinite ratio — fixed by requiring a minimum number of baseline edits. Pages with two edits in a minute against a baseline of one edit in nine minutes produced enormous ratios from trivial counts — fixed by requiring a minimum burst count. And a genuine storm, once detected, re-satisfied the condition on every following edit and produced hundreds of alerts for one event — fixed by a per-page cooldown. Each guard in Section 3.7 corresponds to one of these failures, and each has a test case constructed to trip it.

Memory grew without bound. The detector keeps a timestamp history per page. Wikipedia sees edits to hundreds of thousands of distinct pages per day, and the first version never forgot any of them. Over a long run the per-shard map grew steadily. The fix is a periodic sweep that evicts pages idle for five minutes — their history would have aged out of the ten-minute window anyway, so nothing is lost.

The latency benchmark measured queueing, not handoff. An early version of the latency benchmark used a normal-sized queue. With a producer that pushes faster than the consumer pops, the queue fills, and each item's measured "latency" is dominated by the time it spent sitting in the backlog behind other items rather than by the cost of the handoff itself. The fix was to use a capacity of 2, so the queue is never more than one item deep. That exposed a second problem: with a tiny queue the producer's push often fails and must retry, and if the timestamp was taken before the first attempt, the retry time was counted as latency. The producer now re-stamps on every retry.

The queues overflowed the stack. A four-shard array of 4096-slot queues of 848-byte events is about 13.6 MB. Declared as a local variable in `main`, that exceeds the default 1 MB thread stack on Windows. Both the application and the benchmark keep the shard array in static storage.

The benchmark threads were on the same core. This one I found while re-reading the benchmark source to write this paper, after all the measurements were taken. The harness pins the

producer and consumer to logical processors 0 and 1 so the scheduler cannot migrate them. On the i9-13900H, logical processors 0 and 1 are the two hardware threads of the same physical performance core. They share that core's L1 and L2 caches, which means the specific hardware behavior the padding optimization targets — one core invalidating another core's copy of a cache line — never happened in the benchmark at all. The optimized queue is still faster, but I cannot currently say how much of the 4.8× is due to the cache-line layout and how much to the cheaper memory ordering, and the measurement does not test the cross-core case that the production pipeline actually runs in (where the operating system will generally place five busy threads on five different cores). The fix is to pin the consumer to logical processor 2 instead. I have left the original numbers in this paper, with this explanation, rather than silently replacing them, because the mistake is itself one of the results: a benchmark can run cleanly, produce plausible numbers, and still not be measuring the thing it was written to measure.

The synthetic events were too easy. The pipeline benchmark generates its own events, and to keep the generator simple I gave them nine fields in 182 bytes. Real events, as the live capture later showed, are about 1,300 bytes with a nested metadata block. The benchmark therefore flatters the parser and overstates the pipeline's real throughput, and I did not notice until I put the two numbers next to each other for Section 4.4. The right fix is to drive the benchmark from a real capture file, which the replay mechanism already supports.

6 Conclusion and Future Work

The project achieved its technical goal. The lock-free SPSC ring buffer is 20× faster than a mutex-protected queue in throughput and cuts tail latency by a factor of 13.75, and the complete pipeline ingests a live feed, routes it across four shards, and detects anomalous editing bursts with no event loss. The more durable outcomes, though, were the things the measurements taught me that the implementation alone would not have.

6.1 Lessons

An unoptimized build can hide the entire effect you are measuring. My first benchmark run said the optimized queue was 6% faster than the naive one. Had I stopped there, the honest conclusion would have been that careful memory ordering and cache-line padding were not worth the complexity. The number was wrong because `CMAKE_BUILD_TYPE` was empty and nothing was optimized; with `-O3` the same code showed a 4.8× gap. I now treat the build configuration as part of the measurement itself, and check it before believing any performance number — particularly a negative result, which is exactly the case where it is tempting to accept the number and move on.

A benchmark can be wrong in ways that produce plausible numbers. The build-type mistake announced itself with a surprising result. The core-pinning mistake did not: the numbers looked right, the ratios were in the expected range, and nothing prompted a second look until I re-read the source. The lesson is that a benchmark harness needs the same scrutiny as the code it measures, and specifically that every assumption it encodes about the hardware — what "core 0 and core 1" means on this machine, what the clock resolution is, what the payload looks like — should be checked against the actual machine rather than assumed.

Optimizing a component is not the same as optimizing a system. Measuring the full pipeline showed that producing an event costs roughly 900 times more than handing it off. The queue I spent the most effort on was never the bottleneck, and making it 20× faster moved end-to-end throughput by a few percent. This is Amdahl's law arriving in concrete form: the ceiling on any component's improvement is its share of total time. The lesson I took is to measure the whole path *before* choosing what to optimize, not after — and that the honest framing of this project's result is that it made an already-cheap stage cheaper.

Knowing when performance does not matter is part of engineering judgment. The live stream runs at 43.2 events per second. The pipeline sustains on the order of 100,000. Nothing about this workload requires a lock-free queue; a mutex at 8 million operations per second would have been ample. I think the right conclusion is not that the work was wasted — I understand memory ordering, cache coherence, and measurement methodology far better for having done it — but that "this is faster" and "this project needs it" are separate claims, and conflating them is how effort ends up in the wrong place.

Hardware behavior explains software performance. Before this project, false sharing and memory ordering were things I had read about. Working through why the queue needs exactly a release/acquire pair — not sequential consistency, not nothing — turned an obscure API into the precise vocabulary for stating which guarantee a design actually requires. And discovering that my benchmark had put both threads on one core, and working out why that matters, made cache coherence concrete in a way that reading about it never had.

Designing for testability was the highest-leverage decision. Introducing the `EventSource` abstraction and the capture/replay mechanism turned a system whose behavior depended on live network traffic into one testable deterministically in milliseconds. That single decision produced the detector tests, the benchmark harness, and the ability to measure real traffic reproducibly. It was not a performance optimization, but it did more for the quality of the project than any of them.

6.2 What I would do differently

- **Measure the whole pipeline first.** I built and tuned the queue before I had a working parser, so I had no way of knowing it was 900 times cheaper than the stage feeding it. A rough end-to-end number on day one would have redirected the effort.
- **Benchmark against real data from the start.** The capture mechanism existed; the pipeline benchmark should have used a captured file rather than a synthetic generator with unrealistically small events.
- **Verify the benchmark harness against the hardware.** Check the core topology before pinning threads, check the clock resolution before quoting nanoseconds, and run each configuration several times so that noise is visible.
- **Compare against something real.** A strawman baseline shows an optimization works; only an established implementation shows whether it is competitive.

6.3 Future work

In rough order of how much each would change the paper's conclusions:

1. **Re-pin and re-run the queue benchmarks** with producer and consumer on different physical cores, so that the false-sharing mechanism is actually exercised. Run each configuration ten times and report medians with spread.
2. **Run the pipeline benchmark with the mutex queue swapped in.** This is a single template argument and turns the paper's central claim — that the queue optimization is invisible end to end — from an inference into a measurement.
3. **Add an established SPSC queue to the comparison**, such as `boost::lockfree::spsc_queue` or Folly's `ProducerConsumerQueue`.
4. **Drive the pipeline benchmark from a real capture** of at least several minutes, and report per-shard load balance so the skew introduced by title hashing is visible.
5. **Replace the general-purpose JSON parser** with a targeted extractor for the nine fields actually used, without building a DOM or allocating. Since parsing dominates, this is the one change that would move end-to-end throughput substantially.
6. **Split the naive baseline into two variants** — one with only sequentially-consistent ordering, one with only the shared cache line — to attribute the speedup between the two effects.
7. **Repeat on a weakly-ordered platform** such as an ARM machine to test the prediction that the ordering effect is larger there.

A Reproducing the measurements

The commands below are as run in an MSYS2 UCRT64 shell on Windows, which is the environment of Table 1; on Linux or macOS they are identical. Configure an optimized build and compile all targets:

```
cmake -B build-release -DCMAKE_BUILD_TYPE=Release
cmake --build build-release
```

Run the queue microbenchmarks (throughput and latency), then the end-to-end pipeline benchmark:

```
./build-release/bin/bench_spsc
./build-release/bin/bench_pipeline
```

Run the test suite:

```
cd build-release && ctest --output-on-failure
```

Record 30 seconds of live traffic, then replay it at maximum speed:

```
./build-release/bin/rt_wiki_pipeline --record capture.tsv
./build-release/bin/rt_wiki_pipeline --replay capture.tsv --speed 0
```

Note that build type must be set explicitly. An empty `CMAKE_BUILD_TYPE` produces an unoptimized binary and, as Section 4.6 documents, materially different results. Note also that the thread pinning in `bench_spsc` is Windows-only and, as Section 4.8 documents, places both threads on one physical core on the measurement machine; the numbers in this paper were taken with that configuration.